

SISTEMA COLABORATIVO DE ROBOTS MÓVILES PARA ASISTENCIA DE ORIENTACIÓN EN ENTORNOS INTERIORES CON MÚLTIPLES PISOS

Santiago Hernández Ávila

Jonny Alejandro Mejía León

Modalidad de Proyecto de Grado
(Proyecto de Investigación)

GED — Grupo de Estudio y Desarrollo en Robótica
(Robótica y Sistemas Autónomos)



UNIVERSIDAD SANTO TOMÁS
PRIMER CLAUSTRO UNIVERSITARIO DE COLOMBIA

UNIVERSIDAD SANTO TOMÁS
FACULTAD DE INGENIERÍA ELECTRÓNICA
PROGRAMA DE INGENIERÍA ELECTRÓNICA
Bogotá D.C.
2026

**SISTEMA COLABORATIVO DE ROBOTS MÓVILES PARA ASISTENCIA
DE ORIENTACIÓN EN ENTORNOS INTERIORES CON MÚLTIPLES
PISOS**

Santiago Hernández Ávila

Jonny Alejandro Mejía León

Proyecto de grado presentado como requisito para optar al título de:
Ingeniero Electrónico

DIRECTOR DEL PROYECTO
Ing. Armando Mateus Rojas, MSc.

UNIVERSIDAD SANTO TOMÁS
FACULTAD DE INGENIERÍA ELECTRÓNICA
PROGRAMA DE INGENIERÍA ELECTRÓNICA
Bogotá D.C.
2026

AUTORIDADES ACADÉMICAS

P. NOMBRES COMPLETOS, O.P.

Rector General

P. NOMBRES COMPLETOS, O.P.

Vicerrector Administrativo y Financiero General

P. NOMBRES COMPLETOS, O.P.

Vicerrector Académico General

NOMBRE COMPLETO

Secretaria General

P. NOMBRES COMPLETOS, O.P.

Director de División de Ingenierías

NOMBRE COMPLETO

Secretaria de División de Ingenierías

NOMBRE COMPLETO

Decano Facultad de Ingeniería Electrónica

Nota de aceptación:

Firma del Director

Firma del Jurado 1

Firma del Jurado 2

Bogotá D.C., Colombia
octubre de 2026

ADVERTENCIA

La Universidad Santo Tomás no se hace responsable de las opiniones y conceptos expresados en el trabajo de grado, solo velará por qué no se publique nada contrario al dogma ni a la moral católica y porque el trabajo no tenga ataques personales y únicamente se vea el anhelo de buscar la verdad científica.

AGRADECIMIENTOS

Al ingeniero Armando Mateus Rojas, director del proyecto, por el seguimiento semanal y por insistir en que cada afirmación del documento tuviera detrás un dato que la sostuviera.

Al Grupo de Estudio y Desarrollo en Robótica de la Universidad Santo Tomás, por el espacio de trabajo y por los dos vehículos AWS DeepRacer. Sin ellos el proyecto habría tenido que empezar por construir el hardware.

A los docentes y compañeros de la Facultad de Ingeniería Electrónica que prestaron equipo, acompañaron las jornadas de medición en los pasillos y leyeron borradores de este documento.

A nuestras familias, por el respaldo durante los dos semestres que tomó el trabajo.

Conflicto de intereses

Los autores declaran no tener ningún conflicto de intereses. Los patrocinadores no participaron en el diseño del estudio, la recopilación, el análisis ni la interpretación de los datos publicados, ni en la redacción del documento.

RESUMEN

Orientarse en un edificio de varios pisos es difícil para quien no lo conoce, y el cambio de nivel corta la continuidad del recorrido. Las soluciones robóticas existentes resuelven ese salto de dos maneras: dotando al robot de mecanismos para subir escaleras, o conectándolo al ascensor. Las dos son caras y obligan a intervenir el edificio.

Este trabajo propone y evalúa una alternativa. Dos robots móviles terrestres, uno por planta, se reparten el guiado de una persona mediante un protocolo de relevo en la zona de escaleras. Ningún vehículo cruza de nivel. Un coordinador central asigna el agente según el nivel del origen y del destino. El usuario pide el servicio desde una interfaz web que funciona en cualquier teléfono, sin instalar nada y sin salida a internet.

La evaluación siguió un protocolo escrito antes de instrumentar el sistema, con cuatro métricas de definición operativa y treinta misiones sorteadas en simulación. La campaña resultó válida sin descartes. Permite afirmar, con 95 % de confianza, que la tasa de éxito supera el 70 %.

Sobre los vehículos físicos se integró y se puso en marcha la cadena completa de navegación, y la evaluación experimental continúa en el momento de redactar este documento. De esa integración salió una limitación medida de la plataforma que condiciona la precisión de llegada: el vehículo no admite velocidades por debajo de 0,40 m/s, así que no tiene régimen de aproximación fina.

Palabras clave: robótica móvil colaborativa; sistemas multi-robot; navegación en interiores; protocolo de relevo; interacción humano-robot; ROS 2.

ABSTRACT

Finding one's way inside a multi-storey building is hard for first-time visitors, and changing floors breaks the continuity of the route. Existing robotic solutions address that gap in two ways: by equipping the robot to climb stairs, or by interfacing it with the building lift. Both are expensive and require modifying the infrastructure.

This work proposes and evaluates an alternative. Two ground mobile robots, one per floor, share the task of guiding a person through a handover protocol at the stairwell. Neither vehicle crosses between levels. A central coordinator assigns the agent according to the floor of the origin and the destination. The user requests the service from a web interface that runs on any phone, with no installation and no internet access.

The evaluation followed a protocol written before instrumenting the system, with four operationally defined metrics and thirty randomised missions in simulation. The campaign was valid with no discards. It supports, with 95 % confidence, the claim that the success rate exceeds 70 %.

On the physical vehicles the full navigation chain was integrated and brought up, and the experimental evaluation is still under way at the time of writing. That integration revealed a measured platform limitation which constrains arrival accuracy: the vehicle does not accept commanded speeds below 0.40 m/s, so it has no fine approach regime.

Keywords: collaborative mobile robotics; multi-robot systems; indoor navigation; handover protocol; human-robot interaction; ROS 2.

GLOSARIO

Los términos siguientes se usan desde el resumen y la introducción. El capítulo 2 los desarrolla y añade los demás.

Agente

Cada uno de los dos robots del sistema. Se usa «agente» y no «robot» cuando importa su papel dentro de la coordinación y no su hardware.

AMCL

Algoritmo de localización por filtro de partículas. Combina la odometría con las lecturas del láser para situar al robot dentro de un mapa que ya conoce. No construye el mapa: lo supone dado.

Banda muerta de tracción

Rango de velocidades ordenadas en el que el vehículo no llega a moverse, porque la señal resultante no vence el rozamiento estático. En la plataforma de este trabajo abarca todo lo que está por debajo de 0,40 m/s, y condiciona la aproximación final del vehículo al destino.

Cinemática tipo automóvil (dirección Ackermann)

Modelo de un vehículo que dirige con el eje delantero. Tiene radio de giro mínimo y no puede girar sobre su propio eje. Es lo contrario de la tracción diferencial, que es lo que la mayoría de las herramientas de navegación supone por omisión.

Continuidad del servicio

Propiedad de no interrumpir la asistencia mientras el usuario cambia de nivel. Es el objeto de la pregunta problema.

Localización

Estimación de la posición y la orientación del robot respecto a un marco de referencia fijo.

Mapa de ocupación

Cuadrícula en la que cada celda se declara libre, ocupada o desconocida. Es la representación del edificio con la que funcionan la localización y la planificación.

Mapa de costos (*costmap*)

Cuadrícula derivada de la anterior en la que cada celda lleva un costo de tránsito en lugar de una sola etiqueta. Permite que el planificador prefiera el centro del pasillo sin prohibir los bordes.

Middleware robótico

Capa de software que comunica entre sí los módulos del sistema. Aquí es ROS 2.

Misión

Una solicitud de guiado completa, desde que el usuario elige origen y destino hasta que el sistema la cierra. Es la unidad de medida de la campaña experimental: las métricas se reportan por misión.

Observabilidad

Propiedad de una magnitud física que permite estimarla a partir de las mediciones disponibles. Si en cierta combinación de entorno y sensor una magnitud es inobservable, ningún ajuste de parámetros la recupera, porque la información no está en la señal. El §2.3.4 muestra un caso concreto en un pasillo recto.

Odometría

Estimación del desplazamiento propio del robot a partir de sus propias mediciones. La plataforma de este trabajo no tiene encoders, así que la obtiene comparando barridos sucesivos del láser.

Punto de transferencia

Lugar físico, en este trabajo la escalera, donde un agente entrega el guiado y el otro lo recibe.

Relevo

Traspaso de la responsabilidad de guiar a una persona, de un agente al otro, en un punto acordado de antemano. Es el aporte que el proyecto declara, y lo que sustituye al hardware de transición vertical.

Tasa de éxito

Proporción de misiones que terminan como el protocolo experimental declara que deben terminar. Su definición operativa está en el §3.4.

Tiempo de respuesta

Intervalo entre la solicitud del usuario y el instante en que el robot asignado arranca la tarea.

Zona de transición

Punto del edificio donde se cambia de nivel: escalera o ascensor. En este sistema ningún robot la atraviesa.

ABREVIATURAS

AMCL

Adaptive Monte Carlo Localization — localización adaptativa por método de Monte Carlo.

AWS

Amazon Web Services, fabricante de la plataforma DeepRacer empleada como vehículo.

CONPES

Consejo Nacional de Política Económica y Social (Colombia).

DDS

Data Distribution Service — protocolo de transporte sobre el que ROS 2 implementa sus comunicaciones.

GED

Grupo de Estudio y Desarrollo en Robótica, grupo de investigación en el que se inscribe el proyecto.

GUI

Graphical User Interface — interfaz gráfica de usuario.

HRI

Human-Robot Interaction — interacción humano-robot.

ICONTEC

Instituto Colombiano de Normas Técnicas y Certificación.

IMU

Inertial Measurement Unit — unidad de medición inercial.

JSON

JavaScript Object Notation, formato de intercambio de datos en texto.

MRS

Multi-Robot System — sistema multi-robot.

Nav2

Navigation2, marco de navegación autónoma de ROS 2.

NTC

Norma Técnica Colombiana.

OE

Objetivo específico. Los cuatro del proyecto están en el §1.3.2 y organizan los capítulos 3 y 4.

PID

Control proporcional-integral-derivativo.

PWM

Pulse Width Modulation — modulación por ancho de pulso.

QoS

Quality of Service — políticas de entrega de un canal de comunicación.

RF

Requisito funcional. La numeración RF-*nn* remite a la especificación de requisitos del sistema.

ROS

Robot Operating System. La versión empleada es ROS 2, que no es una actualización de ROS 1 sino un rediseño sobre DDS.

SLAM

Simultaneous Localization and Mapping — localización y mapeo simultáneos.

CONTENIDO

Autoridades académicas	I
Agradecimientos	IV
Resumen	V
Abstract	VI
Glosario	VII
Abreviaturas	IX
Contenido	XI
Lista de figuras	XIV
Lista de tablas	XV
INTRODUCCIÓN	1
1. Contextualización	3
1.1. Planteamiento del problema	3
1.2. Pregunta problema	3
1.3. Objetivos	4
1.3.1. Objetivo general	4
1.3.2. Objetivos específicos	4
2. Marco referencial	5
2.1. Antecedentes	5
2.1.1. Navegación autónoma en interiores	5
2.1.2. Coordinación entre varios robots	6
2.1.3. Movilidad entre pisos	7
2.1.4. Interacción con el usuario	7
2.1.5. El hueco que ocupa este trabajo	8
2.2. Marco conceptual	8
2.2.1. Navegación y percepción	8
2.2.2. Control y locomoción	9

2.2.3.	Comunicación y sistemas distribuidos	10
2.2.4.	Arquitectura del sistema	10
2.2.5.	Interacción humano-robot	11
2.2.6.	Métricas de evaluación	11
2.3.	Marco teórico	12
2.3.1.	Robot móvil, sistema multi-robot y navegación	12
2.3.2.	ROS 2 como capa de integración, en dos distribuciones	12
2.3.3.	Navegación autónoma sobre cinemática tipo automóvil	13
2.3.4.	Localización y mapeo: observabilidad en un corredor uniforme	13
2.3.5.	Simulación multi-robot: aislamiento por espacio de nombres	14
2.3.6.	La interfaz humano-robot como frontera de protocolo	14
2.3.7.	Evaluación experimental: métricas, significancia y falsabilidad	14
2.4.	Marco legal	15
2.4.1.	A alcance de la operación y responsabilidad	15
2.5.	Marco ambiental	16
2.5.1.	Reutilización de hardware existente	16
2.5.2.	Consumo energético y autonomía	16
2.5.3.	Ciclo de vida de las baterías	16
3.	Desarrollo	18
3.1.	Arquitectura funcional del sistema (OE1)	18
3.1.1.	El contrato de interfaces	20
3.1.2.	El planificador, escrito sin ROS a propósito	20
3.1.3.	La asignación de tareas	21
3.2.	Plataforma robótica (OE2)	21
3.2.1.	Por qué DeepRacer y no DonkeyCar	21
3.2.2.	Nav2 sobre un vehículo que no gira sobre su eje	21
3.2.3.	El orden de puesta en marcha	22
3.2.4.	La odometría, que fue el cuello de botella	22
3.2.5.	Los mapas, derivados de medidas y no de SLAM	24
3.2.6.	La banda muerta de tracción, y lo que obliga a configurar	24
3.3.	Interfaz humano-robot (OE3)	25
3.3.1.	Una página web, no una aplicación	25
3.3.2.	Lo que el usuario ve	25
3.4.	Protocolo experimental (OE4)	26
4.	Resultados	27

4.1. Evaluación en simulación	27
4.1.1. Tasa de éxito (RF-23)	27
4.1.2. Los cuatro fallos son un solo modo	28
4.1.3. Tiempo de respuesta (RF-21) y de asignación (RF-22)	28
4.2. Evaluación sobre hardware	29
4.3. Limitaciones de la medición	30
4.3.1. La continuidad entre niveles es un invariante, no un resultado	30
4.3.2. El intervalo de confianza es ancho por diseño	30
4.3.3. El mapa de la campaña leía como libre lo desconocido	30
CONCLUSIONES	32
TRABAJOS FUTUROS	34
Referencias bibliográficas	36

LISTA DE FIGURAS

- 3.1. Arquitectura funcional del sistema colaborativo, tal como quedó modelada en el OE1. Ningún robot atraviesa la zona de transición: lo que la cruza es el usuario, y lo que se traspasa entre agentes es la responsabilidad de guiarlo. 19
- 3.2. Tres recorridos en el hall del piso 2. El círculo es la pose que reporta la localización al terminar; el cuadrado, la posición real medida con flexómetro. La separación entre los dos es el error, y su causa está en la odometría, no en el algoritmo de localización. 23
- 3.3. Mapa de navegación del piso 4, derivado del levantamiento con flexómetro. El pasillo corre 25 m del extremo norte al sur; los salones quedan al este, y el ascensor y la escalera al oeste. 24
- 4.1. Una misión de la campaña en simulación. Se superponen el primer plan que calcula el navegador, la pose estimada a lo largo del recorrido y la odometría cruda. La separación creciente entre las dos últimas es la deriva que la localización corrige contra el mapa. 29

LISTA DE TABLAS

4.1. Tasa de éxito de la campaña, con intervalo de confianza de Wilson.	28
4.2. Tiempo de respuesta, en escalones del reloj de simulación.	28

INTRODUCCIÓN

Orientarse dentro de un edificio grande no es igual de fácil para todo el mundo. Quien llega por primera vez construye un mapa mental a partir de información parcial, y lo va corrigiendo mientras camina. Ese esfuerzo crece cuando el espacio tiene pasillos repetidos, pocos puntos de referencia o señalización escasa. Y crece más cuando hay que cambiar de piso, porque la escalera corta la continuidad de lo que la persona lleva recorrido. Jiménez Muñoz lo señala al construir una arquitectura cognitiva que mete al usuario dentro de un sistema de mapas de varios niveles: la escalera es el punto donde la representación mental del recorrido deja de servir [1].

La robótica móvil colaborativa lleva años resolviendo tareas parecidas. El reparto de trabajo entre varios robots tiene base teórica desde que Gerkey y Mataric [2] formalizaron la asignación de tareas en sistemas multi-robot. Y Varghese et al. [3] muestran que varios agentes pueden mapear y localizarse de forma cooperativa. Casi todos esos sistemas operan, sin embargo, en una sola planta.

El cambio de nivel se ha abordado por dos caminos, y los dos pasan por el hardware. Kim et al. [4] combinan un mapa por piso con un grafo topológico y hacen que el robot use el ascensor. Song et al. [5] van más lejos: automatizan ese uso instalando dispositivos conectados en el edificio. La otra opción es dotar al robot de mecanismos para subir escaleras. Ambas encarecen la plataforma y exigen intervenir la infraestructura.

Este trabajo propone otra. En vez de un robot que cruza de piso, se usan **dos robots, cada uno dedicado a un nivel**. El usuario sube por su cuenta y un segundo agente lo recibe arriba. El cambio de piso deja de ser un problema de locomoción y pasa a ser uno de coordinación. Dos agentes se reparten una tarea y se la pasan en un punto acordado. A ese traspaso lo llamamos *relevo*, y es el aporte que el proyecto declara.

La pregunta que organiza el trabajo es si ese esquema funciona. No basta con que los robots se muevan. Hay que demostrar que la asistencia no se interrumpe cuando el usuario cambia de nivel, y medirlo con criterios fijados antes de correr los experimentos. Por eso el proyecto dedica tanto espacio al protocolo experimental como a la implementación.

El sistema se construyó sobre dos vehículos AWS DeepRacer y software libre. La coordinación corre en un nodo central que habla con cada agente a través de ROS 2. Macenski et al. [6] lo describen como un rediseño sobre DDS, no como una actualización de su antecesor. El usuario pide el servicio desde una página web que funciona en cualquier teléfono, sin instalar nada y sin salida a internet. La

evaluación se hizo primero en simulación, con una campaña de treinta misiones sorteadas, y después sobre los vehículos reales en los pasillos del edificio.

El documento sigue ese orden. El capítulo 1 plantea el problema y los objetivos. El 2 reúne el marco conceptual, teórico, legal y ambiental. El 3 describe cómo se construyó el sistema, organizado por objetivo específico. El 4 presenta los resultados y, con el mismo énfasis, sus límites. Los capítulos 5 y 6 cierran con las conclusiones y las líneas que quedan abiertas.

1. CONTEXTUALIZACIÓN

1.1 Planteamiento del problema

Orientarse en un edificio desconocido exige construir y actualizar una representación mental del espacio. El proceso se complica cuando el entorno tiene visibilidad limitada, estructuras repetidas y varios puntos de decisión. Omer y Monteriu [7] describen esos espacios compartidos con personas como el escenario que más exige a un robot. Vale también para quien camina sin conocerlos.

Las edificaciones de varios pisos añaden una dificultad propia. El cambio de nivel rompe la continuidad espacial y obliga a integrar información vertical en el recorrido. Es el mismo problema que Kim et al. [4] resuelven para un robot, con un grafo topológico entre plantas.

La robótica móvil colaborativa ha avanzado en exploración, logística y cobertura distribuida. La asignación de tareas entre agentes tiene una formalización aceptada desde el trabajo de Gerkey y Mataric [2]. Pero la mayoría de esos desarrollos se evaluó en entornos de una sola planta, donde la segmentación vertical no es el problema central. Los casos de limpieza doméstica y de almacén que documenta Nair [8] son representativos de ese escenario plano.

Resolver el cambio de piso con un solo robot resulta caro. Exige mecanismos de locomoción capaces de subir escaleras, o intervenir el sistema de control del ascensor. Song et al. [5] lo hacen instalando dispositivos conectados en la propia cabina. Las dos opciones elevan el costo del hardware y obligan a modificar el edificio.

De ahí el planteamiento de este trabajo. El problema no es cómo hacer que un robot suba, sino **cómo lograr que dos robots sencillos se repartan el recorrido y se pasen al usuario sin que la asistencia se interrumpa**. La comunicación entre agentes sustituye al hardware de transición vertical.

1.2 Pregunta problema

¿Es viable mantener la continuidad de la asistencia de orientación a una persona que cambia de nivel en un edificio, mediante un esquema de relevo entre dos robots móviles dedicados cada uno a una planta, sin que ninguno de ellos atraviese la zona de transición

vertical?

La pregunta tiene dos mitades y las dos se responden en el capítulo 4. La primera es de funcionamiento: si el relevo ocurre. La segunda es de medida: con qué incertidumbre se puede afirmar.

1.3 Objetivos

1.3.1 Objetivo general

Implementar un sistema colaborativo de dos robots móviles terrestres para la guía coordinada y eficiente de asistencia a visitantes en entornos interiores con múltiples pisos, mediante comunicación inter-robot, asignación dinámica de tareas y una interfaz móvil de interacción usuario-robot.

1.3.2 Objetivos específicos

- OE1.** Modelar la arquitectura funcional del sistema colaborativo de dos robots móviles, definiendo requerimientos, escenarios de operación con múltiples pisos, estrategia de asignación de tareas y esquema de comunicación inter-robot.
- OE2.** Desarrollar una plataforma robótica móvil basada en dos vehículos tipo DonkeyCar, integrando los módulos de locomoción, sensado, procesamiento y comunicación necesarios para la operación colaborativa en entornos interiores.
- OE3.** Programar una interfaz móvil para interacción humano-robot (HRI) basada en la selección de localizaciones de interés de origen y destino.
- OE4.** Evaluar el desempeño del sistema mediante pruebas experimentales en un entorno interior controlado, utilizando métricas como tiempo de respuesta, tiempo de asignación de robot, tasa de éxito en la entrega de asistencia y continuidad del servicio entre niveles.

Estos cuatro objetivos organizan el resto del documento. El capítulo 3 dedica una sección a cada uno, y el 4 reporta lo que se midió para cada uno.

2. MARCO REFERENCIAL

Este capítulo reúne cinco secciones. Los antecedentes revisan qué se ha hecho antes y dónde queda el hueco que este trabajo ocupa. El marco conceptual fija qué significa cada término técnico del documento. El teórico explica las decisiones de arquitectura y por qué se tomaron. El legal sitúa el trabajo frente a la normativa que le aplica. El ambiental recoge las consideraciones de consumo y de ciclo de vida del hardware.

2.1 Antecedentes

La revisión se ordena en cuatro líneas: navegación autónoma en interiores, coordinación entre varios robots, movilidad entre pisos e interacción con el usuario. Cada una resuelve bien una parte del problema. Al final se señala lo que ninguna resuelve, que es donde entra este trabajo.

2.1.1 Navegación autónoma en interiores

Sojan et al. [9] integran percepción, localización y planificación sobre ROS 2 para desplazamiento autónomo en interiores dinámicos. Su aporte es consolidar el flujo completo —mapa, localización por filtro de partículas y planificación— sobre una representación plana del espacio. Es la arquitectura que este proyecto toma como punto de partida en cada agente.

Varghese et al. [3] llevan ese mismo problema a un escenario cooperativo: varios agentes construyen un mapa compartido y mantienen consistente su localización. El énfasis se desplaza de la autonomía individual a la construcción colectiva del entorno. La diferencia con el trabajo anterior está en el alcance, no en la técnica: las dos propuestas usan representaciones bidimensionales y se apoyan en ROS.

Existe una línea distinta que resuelve la navegación con aprendizaje en lugar de con planificación explícita. Bojarski et al. [10] mostraron que una red neuronal entrenada de extremo a extremo, solo con imágenes de cámara y el ángulo de dirección del conductor, aprende a conducir sin que nadie programe el seguimiento de carril. Zain [11] aplica una idea parecida a la evasión de obstáculos, fusionando sensores con una red convolucional, y López Velásquez et al. [12] combinan aprendizaje profundo con lógica difusa para obtener autonomía reactiva.

Este proyecto no sigue esa vía, y conviene decir por qué. Entrenar cualquiera de esos modelos exige un conjunto de datos etiquetados del edificio concreto, que no existe y que habría que levantar. Además, el aporte declarado aquí no está en la percepción sino en la coordinación: dos agentes que se reparten un recorrido. Con planificación clásica cada decisión del robot es inspeccionable, y eso importa cuando hay que explicar un fallo. Una red entrenada daría mejor desempeño reactivo a cambio de no poder decir por qué hizo lo que hizo.

2.1.2 Coordinación entre varios robots

Las estrategias de coordinación de la literatura revisada se diferencian por el nivel en el que cooperan. Nair [8] funda la cooperación en el intercambio de información sensorial: los robots comparten lo que perciben para mejorar el desempeño del conjunto. Roszkowska et al. [13] proponen en cambio una estructura jerárquica que supervisa el desplazamiento simultáneo de varios agentes, con el fin de prevenir conflictos. Rudrusamy et al. [14] resuelven el problema desde el canal de comunicación, con un esquema de control por difusión y seguimiento visual.

Xu et al. [15] aportan una arquitectura modular sobre ROS para asignación cooperativa de tareas, y Zhan et al. [16] atacan la resiliencia del sistema cuando un nodo falla. El marco teórico de la asignación de tareas lo fija Gerkey y Mataric [2], cuya taxonomía permite clasificar cualquier esquema de reparto según cuántas tareas acepta un robot, cuántos robots requiere una tarea y con cuánta anticipación se decide. En el §2.3.1 se usa esa taxonomía para situar el esquema de este trabajo.

El antecedente más cercano a la propuesta de este proyecto es Srivastava et al. [17], que coordinan recogida y entrega entre varios robots mediante planificación con relevos. Allí lo que se traspasa de un agente a otro es un paquete. Aquí es una persona, y esa diferencia no es menor: un paquete espera indefinidamente en el punto de transferencia, y una persona no.

Un antecedente de alcance comparable al de este trabajo, por tratarse también de un proyecto de grado de ingeniería, es Piguave Toala y Ronquillo Manosalvas [18], que implementan un sistema multi-robot de código abierto sobre ROS 2 para ambientes colaborativos.

Lo que comparten todos estos trabajos es el escenario: operan en un solo nivel físico, sin segmentación vertical.

2.1.3 Movilidad entre pisos

El cambio de nivel se ha abordado sobre todo desde la logística y la exploración. Kim et al. [4] resuelven la planificación tridimensional combinando un mapa por cada piso con un grafo topológico que los conecta, e integran además la interacción con el ascensor. Song et al. [5] se concentran justamente en eso último: automatizar el uso del elevador mediante dispositivos conectados instalados en el edificio.

Luo et al. [19] plantean la pregunta un paso antes —ascensor o escalera— y la resuelven con aprendizaje por refuerzo jerárquico, de modo que el robot elige el medio de transición según el tránsito que encuentra. En otra dirección, Chen et al. [20] estructuran el entorno con una descomposición piso-escalera que extiende las técnicas bidimensionales a configuraciones de varios niveles, y Gong et al. [21] incorporan conciencia de piso a la navegación hacia objetos sin entrenamiento previo del escenario.

Las cuatro propuestas resuelven la movilidad entre pisos desde ángulos complementarios: la infraestructura física, el modelado topológico y la decisión adaptativa. Las cuatro comparten un supuesto: que el robot cruza. Todas dotan al agente de la capacidad de cambiar de nivel, sea subiendo o sea accionando el ascensor, y todas se orientan a entrega o exploración antes que a asistencia a personas.

2.1.4 Interacción con el usuario

Jiménez Muñoz [1] integra al usuario dentro del propio sistema de mapas, combinando representaciones semánticas con reconocimiento de personas para localizar y seguir a quien acompaña, en una plataforma asistencial de varios niveles. Es el antecedente más próximo en intención: trata al humano como parte del problema de navegación y no como un destinatario externo.

Li y Gurocak [22] abordan la otra cara, la seguridad: modifican el algoritmo Bug con sensores de proximidad para reducir las colisiones entre un robot colaborativo y las personas que comparten su espacio.

En cuanto a dónde debe ejecutarse el cómputo, Kehoe et al. [23] revisan las arquitecturas que delegan procesamiento a servidores remotos, y Tahir y Parasuraman [24] las que lo acercan al borde de la red. Malavolta et al. [25] miden el costo real de esa delegación sobre WiFi en robots terrestres, que es el caso de este proyecto. Sus resultados pesaron en una decisión de diseño concreta: aquí la navegación corre completa a bordo de cada vehículo, y lo único centralizado es la coordinación, que mueve mensajes pequeños. La interfaz, además, debe funcionar sin salida a internet, porque la red del edificio no siempre la tiene.

2.1.5 El hueco que ocupa este trabajo

La literatura cubre bien cada componente por separado. Hay flujos de navegación completos sobre ROS, hay esquemas de coordinación multi-robot a distintos niveles, hay soluciones de movilidad vertical y hay trabajo sobre la integración del usuario. Lo que no aparece es la combinación que aquí se propone: **resolver la discontinuidad entre pisos sin que ningún robot la atraviese**. El recorrido se reparte entre dos agentes dedicados, y la persona se traspasa en la zona de transición.

La diferencia es económica antes que algorítmica. Los trabajos de movilidad vertical compran la continuidad con hardware —mecanismos de locomoción o intervención del ascensor—. Este proyecto la compra con comunicación, que es lo que los trabajos de coordinación ya saben hacer, aplicada a un problema donde nadie la había puesto.

2.2 Marco conceptual

Aquí se definen los términos necesarios para leer el resto del documento. El lector supuesto tiene formación en ingeniería electrónica. Las seis categorías son las del anteproyecto, y dentro de cada una se añaden los términos que aparecieron durante la implementación.

2.2.1 Navegación y percepción

Se llama **navegación autónoma** al desplazamiento de un robot sin intervención humana. No es una sola capacidad sino tres trabajando juntas: percibir, saber dónde se está y decidir hacia dónde moverse. La primera pieza de «saber dónde se está» es la **odometría**, que estima la posición a partir de las mediciones del propio robot, sean de sus actuadores o de sus sensores. Esa estimación se va desviando con el recorrido, así que hay que corregirla contra algo externo. De eso se encarga **AMCL**, un algoritmo de localización por filtro de partículas. Combina la odometría con las lecturas del láser y sitúa al robot dentro de un mapa que ya conoce.

Hay un concepto que decide si todo lo anterior puede funcionar, y es la **observabilidad** de un estado. Una magnitud es observable cuando se puede estimar a partir de las mediciones disponibles. La propiedad depende de la combinación de entorno y sensor, no solo del sensor: una misma magnitud puede ser observable en un sitio e inobservable en otro. Y cuando es inobservable, ningún ajuste de parámetros la recupera, porque la información no está en la señal. El §2.3.4 muestra el caso concreto que este proyecto encontró.

El entorno se representa de dos formas distintas y complementarias. El mapa de ocupación dice qué celdas están libres y cuáles no. El **mapa de costos** o *costmap* va un paso más allá: asigna a cada celda un costo de tránsito en lugar de una sola etiqueta. Con eso el planificador puede preferir el centro del pasillo sin que los bordes queden prohibidos. Hay una capa global, que cubre todo el mapa, y una local, limitada al entorno inmediato del robot.

Sobre esa representación se calcula la trayectoria. En un vehículo que gira con radio mínimo y puede ir hacia adelante y hacia atrás, las trayectorias óptimas son las del **modelo de movimiento Reeds-Shepp**. El punto donde el vehículo invierte el sentido de la marcha se llama **cúspide**. Contarlas dice cuántas maniobras hizo falta. Al final del recorrido interviene el **verificador de meta** o *goal checker*. Decide si el robot llegó, comparando su posición —y a veces su orientación— contra una tolerancia.

Queda un término que es del edificio y no del robot: la **zona de transición**, el punto donde se cambia de nivel. En este sistema es la escalera, y ningún vehículo la atraviesa.

2.2.2 Control y locomoción

La **cinemática de robots móviles** es el modelo matemático que relaciona las velocidades de los actuadores con el movimiento del vehículo. La plataforma de este trabajo sigue un **modelo cinemático tipo automóvil**, también llamado **dirección Ackermann**. Son cuatro ruedas, dirección en el eje delantero y una relación entre el ángulo del servomotor y el radio de giro. Lo que ese modelo garantiza es que las ruedas interior y exterior giren en torno al mismo centro instantáneo de rotación, de modo que no derrapan. Lo que impone, y condiciona todo lo demás, es un radio de giro mínimo: el vehículo no puede girar sobre su propio eje.

Esa restricción obliga a elegir bien los dos componentes que calculan y siguen la trayectoria. La planificación corre a cargo del **planificador híbrido A***, que busca rutas trabajando sobre posición y orientación a la vez. Un planificador de cuadrícula no sirve aquí: ignora la orientación, así que propone giros que el vehículo no puede ejecutar. El seguimiento lo hace un **controlador de persecución pura regulada**. Persigue un punto móvil sobre la trayectoria y regula la velocidad según la curvatura y la cercanía de obstáculos.

Las órdenes de velocidad llegan a los actuadores como señales de **PWM**, modulación por ancho de pulso: controla motores y servomotores variando el ciclo de trabajo. Su regulación fina se hace con **control PID**, proporcional-integral-derivativo. Entre la orden y el movimiento real aparece un fenómeno que este trabajo tuvo que medir: la **banda muerta de tracción**. Es el rango de velocidades comandadas en el que el vehículo no llega a moverse, porque la señal resultante no vence el rozamiento estático.

2.2.3 Comunicación y sistemas distribuidos

Un **middleware robótico** es la capa de software que comunica entre sí los módulos del sistema. Aquí es **ROS**, *Robot Operating System*, que ofrece nodos, tópicos de publicación y suscripción, servicios de petición y respuesta, y acciones [6]. Por debajo, ROS 2 implementa todo eso sobre **DDS**, un protocolo de transporte que descubre solo a los participantes de la red.

De ese mecanismo de descubrimiento salen dos conceptos que en la práctica deciden quién habla con quién. El **dominio ROS** es un número que segmenta el descubrimiento: dos procesos con dominio distinto no se ven entre sí, salvo que se configure aparte. El **espacio de nombres** o *namespace* es un prefijo que se antepone a los tópicos, servicios y acciones de un nodo. En este trabajo es lo que separa a los dos agentes dentro de un mismo dominio.

De los cuatro patrones de comunicación, el que sostiene el sistema es la **acción**, pensada para tareas largas. Tiene un objetivo, retroalimentación periódica y un resultado final, y se puede cancelar. El nodo que la ofrece es el **servidor de acción** y el que la invoca, el **cliente**. El coordinador de este proyecto es las dos cosas a la vez: cliente ante cada agente y servidor ante la interfaz. Para que un servidor acepte cancelaciones tiene que registrar un **callback de cancelación**. Si no lo hace, las rechaza todas. Y lo hace en silencio. Cada canal lleva además sus políticas de entrega, que se llaman **calidad de servicio** o QoS y se configuran por separado.

La interfaz web no habla ROS, así que entre las dos hay un traductor: el **puente rosbridge**, un servidor que convierte entre el protocolo nativo de ROS 2 y JSON sobre WebSocket. Es lo que permite que una página web opere el sistema sin instalar nada.

2.2.4 Arquitectura del sistema

La **asignación de tareas** es el reparto de actividades entre los robots según su disponibilidad y su ubicación [2]. A veces la tarea no cabe en un solo agente, porque atraviesa la frontera entre dos niveles. Entonces el reparto toma la forma de un **protocolo de relevo**: el traspaso de responsabilidad de un agente a otro en puntos definidos. Ese punto físico, que en este edificio es la escalera, se llama **punto de transferencia**.

Tres piezas más completan la arquitectura. La **máquina de estados de misión** es un conjunto de valores ordenados que describe en qué punto va una misión; se publican para que la interfaz y el registro de evaluación lo sepan. El **planificador puro** calcula los tramos de la misión a partir del catálogo de puntos y del par origen-destino. Se llama «puro» porque no depende del middleware ni de ningún estado externo. Esa independencia se paga sola: permite verificarlo con cientos de casos

en milisegundos, sin levantar el simulador. Y el **registro de misión** es un documento estructurado que se compone a partir de la grabación de cada ejecución y se valida contra un esquema. Es la evidencia primaria del experimento.

2.2.5 Interacción humano-robot

La **HRI** —*Human-Robot Interaction*— es el campo que estudia el diseño y la evaluación de sistemas robóticos pensados para interactuar con personas [1]. Su cara visible aquí es una **interfaz gráfica de usuario** o GUI, el entorno visual donde la persona y la máquina se encuentran. El acto que la pone en marcha es la **solicitud de servicio**: la petición del usuario para recibir guiado.

Dos decisiones de diseño merecen nombre propio porque se tomaron después de ver el sistema en uso. La **interfaz responsiva de selección** permite buscar un destino escribiendo su nombre, sin perder la opción de ver el catálogo completo. Con diez destinos la diferencia es pequeña; con cien no lo sería. Y la **retroalimentación literal** es un principio: el texto que ve el usuario lo redacta el componente que conoce el estado del sistema, y la interfaz lo muestra tal cual. Así no puede contradecir al coordinador.

2.2.6 Métricas de evaluación

Las cuatro métricas del proyecto son las del anteproyecto. El **tiempo de respuesta** es lo que pasa entre la solicitud del usuario y el momento en que el robot arranca la tarea. El **tiempo de asignación** es lo que pasa entre aceptar la solicitud y designar qué agente la atiende. La **tasa de éxito** es la proporción de solicitudes que terminan bien. Y la **continuidad de servicio** es la capacidad de mantener la asistencia mientras el usuario cambia de nivel.

Medir no es solo tomar el dato, y por eso hacen falta cuatro conceptos más, los cuatro aprendidos durante la evaluación. El **intervalo de confianza de Wilson** estima el intervalo de una proporción. Es más fiable que la aproximación normal cuando la muestra es pequeña o la proporción se acerca a 0 o a 1. Una campaña de treinta misiones cae justo en ese caso. La **falsabilidad** de un criterio es la propiedad de que alguna ejecución del sistema pueda, al menos en principio, incumplirlo. Un criterio que no puede fallar no aporta evidencia, por muchas veces que se verifique. Un **invariante estructural** es una propiedad que se cumple siempre por el diseño del sistema. No es un resultado experimental, y no debe presentarse como tal. Y la **cota superior por misión** resuelve qué hacer cuando el instrumento no tiene resolución para dar el valor exacto. Se informa el límite que sí se puede garantizar.

2.3 Marco teórico

2.3.1 Robot móvil, sistema multi-robot y navegación

El sistema lo forman dos robots móviles terrestres. Cada uno atiende un nivel del edificio y los dos colaboran, de modo que juntos constituyen un sistema multi-robot, en el sentido que le dan Varghese et al. [3]. La tarea que ejecutan es guiar a una persona desde un origen hasta un destino, y esa persona elige el destino en una interfaz móvil. Eso convierte al sistema en un caso de interacción humano-robot, el campo en el que Jiménez Muñoz [1] sitúa al usuario dentro del propio sistema de mapas en lugar de tratarlo como un destinatario externo.

La navegación de cada agente se apoya en tres pilares. La localización determina dónde está el robot y hacia dónde mira. La planificación de trayectorias calcula una ruta que el vehículo pueda ejecutar. El control de movimiento genera las señales que la recorren. Los tres se alimentan de la percepción del entorno, que aquí aporta un sensor láser de barrido plano.

2.3.2 ROS 2 como capa de integración, en dos distribuciones

Todos los componentes se comunican mediante ROS 2, cuyo diseño y arquitectura documentan Macenski et al. [6]. El sistema usa dos distribuciones a la vez. La simulación corre sobre una porque el paquete que integra física y simulación no está liberado para versiones posteriores. Los vehículos corren sobre otra, la que soporta el sistema operativo de sus tarjetas.

Esa convivencia impone una restricción que condiciona toda la arquitectura. El tipo de mensaje con que el coordinador manda a los agentes está definido de forma distinta en cada distribución. Un coordinador compilado en una no puede, por tanto, mandar a un agente compilado en la otra. Dicho de otro modo: que el código fuente sea portable entre distribuciones no significa que los mensajes sean compatibles al ejecutarse. La documentación general del middleware no advierte esa diferencia. Para sortearla, el proyecto mantiene una variante de configuración por distribución, derivada de una base común.

Sobre esa capa, la coordinación es centralizada. Un solo nodo coordinador actúa como cliente de acción frente a cada agente y como servidor de acción frente a la interfaz. En la taxonomía de Gerkey y Mataric [2] eso corresponde a una asignación centralizada e instantánea. El coordinador elige el agente al aceptar la solicitud, según el nivel del origen y del destino, y los agentes no negocian entre sí. Con solo dos agentes, y con el nivel decidiendo cuál responde, un esquema de subasta añadiría complejidad sin dar flexibilidad.

2.3.3 Navegación autónoma sobre cinemática tipo automóvil

Cada agente navega con el marco de navegación estándar de ROS 2 que es el flujo que Sojan et al. [9] integran para interiores dinámicos. La plataforma empleada sigue un modelo cinemático tipo automóvil: no es holonómica y no puede girar sobre su propio eje. El problema es que la configuración por omisión de ese marco asume tracción diferencial, o sea un robot que sí gira en el sitio. Por eso se sustituyen dos de sus tres capas:

- **Planificación global.** Se usa un planificador híbrido A* con modelo Reeds-Shepp. Admite marcha atrás y genera las maniobras de inversión que el vehículo necesita para alcanzar orientaciones a las que no puede llegar de frente.
- **Control de seguimiento.** Se usa un controlador de persecución pura regulada, con marcha atrás habilitada, que ejecuta las cúspides que el planificador propuso.

Esta configuración tiene un límite que el desarrollo permitió caracterizar. El controlador acorta su distancia de anticipación cuando el vehículo se acerca a una cúspide muy próxima a la meta. Por debajo de cierto umbral geométrico, una guarda de seguridad del propio controlador anula la curvatura calculada. El resultado es que el vehículo sigue invirtiendo la marcha pero deja de dirigir, y maniobra decenas de veces en el mismo punto. No falla el planificador ni la coordinación: falla un controlador de referencia trabajando en el borde de su dominio de validez.

2.3.4 Localización y mapeo: observabilidad en un corredor uniforme

La localización usa un filtro de partículas sobre el mapa del entorno. El mapa, a su vez, se construye con odometría láser, que compara barridos sucesivos para deducir cuánto se movió el robot.

Ese método tiene un límite en corredores rectos y de sección uniforme: la posición **longitudinal** del robot no es observable. La razón es sencilla. Cada barrido es prácticamente idéntico al anterior a lo largo del eje del pasillo. El estimador devuelve entonces un desplazamiento cercano a cero, y acierta, porque desde el sensor no ha cambiado nada.

Conviene separar dos cosas que se parecen y no lo son. Un **error de calibración** se corrige ajustando parámetros. Una **inobservabilidad estructural** no, porque la información no está en la señal. Por eso el criterio para aceptar un mapa no es un porcentaje fijo de cobertura. Lo que se exige es que el entorno aporte estructura no uniforme dentro del alcance del sensor durante todo el recorrido.

2.3.5 Simulación multi-robot: aislamiento por espacio de nombres

La simulación levanta un proceso de simulación por robot, pero un solo dominio DDS para los dos. Los agentes se separan por espacio de nombres, incluido el marco de referencia de su mapa.

La alternativa evidente sería dar a cada robot un dominio distinto. No sirve: sin configuración adicional, eso corta el descubrimiento entre el coordinador y los agentes. Separar por dominios y comunicar a los agentes resultan, con la configuración por omisión, mutuamente excluyentes.

2.3.6 La interfaz humano-robot como frontera de protocolo

La interfaz web habla con el middleware a través de un puente que expone las acciones de ROS 2 como JSON sobre WebSocket. Ese protocolo no es el que implementa la librería de cliente más difundida del ecosistema web de ROS. La de esa librería corresponde a la generación anterior del middleware y no es compatible con las acciones de ROS 2. Por eso la interfaz de este trabajo es un cliente propio, escrito directamente sobre el protocolo del puente. Tampoco usa dependencias externas, porque el requisito exige que funcione sin salida a internet.

Un segundo principio rige la interfaz: no redacta lógica de negocio. El texto que lee el usuario en cada estado lo escribe el coordinador, y la interfaz lo muestra literal. Es separación de responsabilidades entre el dominio y la presentación. Si el navegador interpretara el estado para decidir qué texto mostrar, se duplicaría en el cliente una lógica que ya vive en el planificador, y las dos copias acabarían divergiendo.

2.3.7 Evaluación experimental: métricas, significancia y falsabilidad

La evaluación se apoya en cuatro métricas con definición operativa: tiempo de respuesta, tiempo de asignación, tasa de éxito y continuidad del servicio entre niveles. Se miden sobre una campaña de misiones sorteadas, con el protocolo fijado *antes* de instrumentar el sistema. Dos principios gobiernan su tratamiento.

El primero es estadístico. Con muestras del orden de treinta observaciones, el intervalo de confianza adecuado para una proporción es el de Wilson, no la aproximación normal, que en ese rango subestima el error. La diferencia no es cosmética: decide si lo que se puede afirmar es una cota inferior sobre la tasa de éxito o un valor puntual. Con esa muestra, solo la cota se sostiene.

El segundo es metodológico, y se enuncia así: **un criterio de aceptación solo es evidencia si**

puede fallar. Durante el desarrollo apareció un caso concreto. El criterio de la variable de respuesta principal estaba definido de forma que ninguna ejecución podía incumplirlo. Los dos únicos puntos del código que producen un valor de incumplimiento ocurren fuera de la ventana en que el criterio se evalúa. Treinta verificaciones satisfactorias de un criterio así no demuestran que el sistema cumpla la propiedad. Demuestran que el criterio está mal planteado.

La corrección no consistió en inventar una ruta de fallo. Eso habría sido instrumentar el experimento para que pareciera más informativo de lo que es. Lo que se hizo fue reformular la afirmación como un invariante estructural del diseño, y respaldarla con una magnitud que sí varía y sí se puede reportar.

2.4 Marco legal

El proyecto se desarrolla dentro de la política pública colombiana de transformación digital y de ciencia, tecnología e innovación. Dos documentos la enmarcan. El CONPES 3975 [26] fija la Política Nacional para la Transformación Digital e Inteligencia Artificial. El CONPES 4069 [27] establece la Política Nacional de Ciencia, Tecnología e Innovación. Ambos piden a la academia que desarrolle prototipos con herramientas de la Cuarta Revolución Industrial, entre ellas la robótica autónoma, y que lo haga con diseño local e impacto social.

Este trabajo encaja en esa línea. Es un prototipo de asistencia robótica construido sobre plataformas de bajo costo y software abierto, dentro de las limitaciones presupuestales de un entorno universitario.

La presentación del propio documento sigue la norma NTC 1486 del ICONTEC [28], que regula cómo se presentan las tesis y los trabajos de grado y que la plantilla institucional declara como su base.

2.4.1 Alcance de la operación y responsabilidad

El alcance del proyecto excluye de forma explícita que los robots interactúen mecánicamente con el edificio. No accionan puertas ni botones de ascensor, no transportan carga y no cruzan las zonas de transición vertical. Esa delimitación se adoptó por razones técnicas y de costo, pero tiene además una consecuencia normativa favorable: evita intervenir sistemas sujetos a regulación específica de seguridad en edificaciones.

2.5 Marco ambiental

El impacto ambiental del prototipo se concentra en tres frentes: el hardware que emplea, la energía que consume y el destino de sus componentes al final de la vida útil.

2.5.1 Reutilización de hardware existente

Las dos plataformas robóticas son equipo institucional que ya existía. El proyecto no las compró, y el presupuesto del anteproyecto las contabiliza como uso y desgaste. La decisión se tomó por razones económicas, pero reduce también la huella material del trabajo: no entran plataformas nuevas al inventario y se alarga la vida útil de equipos que estaban subutilizados.

Lo mismo ocurre con el entorno de evaluación. Es el propio edificio de la universidad, sin construir infraestructura dedicada ni modificar la existente. El sistema opera sobre la planta tal como está.

2.5.2 Consumo energético y autonomía

Cada vehículo lleva dos fuentes de alimentación independientes, una para cómputo y otra para tracción. Una corrida de evaluación dura decenas de segundos, y una sesión de campo completa gasta una fracción de la carga disponible.

De ahí sale un dato que conviene señalar porque va contra la intuición. El consumo de los vehículos durante toda la campaña es marginal frente al de la estación de trabajo que ejecuta la simulación. Esa estación corre dos instancias del simulador a la vez, y de forma sostenida. Es decir: **en un proyecto de robótica de esta escala, el mayor consumo energético no está en los robots sino en la simulación**. Ese hecho respalda además la estrategia de verificación que se adoptó, en la que la lógica de coordinación se comprueba como función pura, sin simulador, en milisegundos y con consumo despreciable.

2.5.3 Ciclo de vida de las baterías

Las baterías de polímero de litio que alimentan la tracción son el componente más sensible del sistema desde el punto de vista ambiental. Contienen litio y se degradan con los ciclos de carga. Al final de su vida útil deben ir a los canales de disposición de residuos electrónicos de la institución, nunca a la basura doméstica.

Durante el desarrollo se observó además que el nivel de carga cambia el comportamiento del vehículo de forma medible. Una misma orden de tracción produjo velocidades distintas en corridas consecutivas, y la batería fue el primer sospechoso. Esa dependencia obliga a anotar el nivel de carga antes y después de cada medición. Es buena práctica experimental y, de paso, desincentiva el uso de baterías degradadas.

3. DESARROLLO

El sistema se construyó en cuatro frentes, uno por objetivo específico. Cada sección describe qué se hizo, qué decisión se tomó y por qué. Las decisiones que hubo que revisar aparecen con su corrección, porque el camino que se descartó explica el que quedó.

3.1 Arquitectura funcional del sistema (OE1)

La figura 3.1 resume el sistema completo. Están la interfaz por la que entra la solicitud, el coordinador que la reparte, los dos agentes con su pila de navegación y la zona de transición donde ocurre el relevo. El protocolo de la parte inferior es la secuencia que el §3.1.2 implementa.

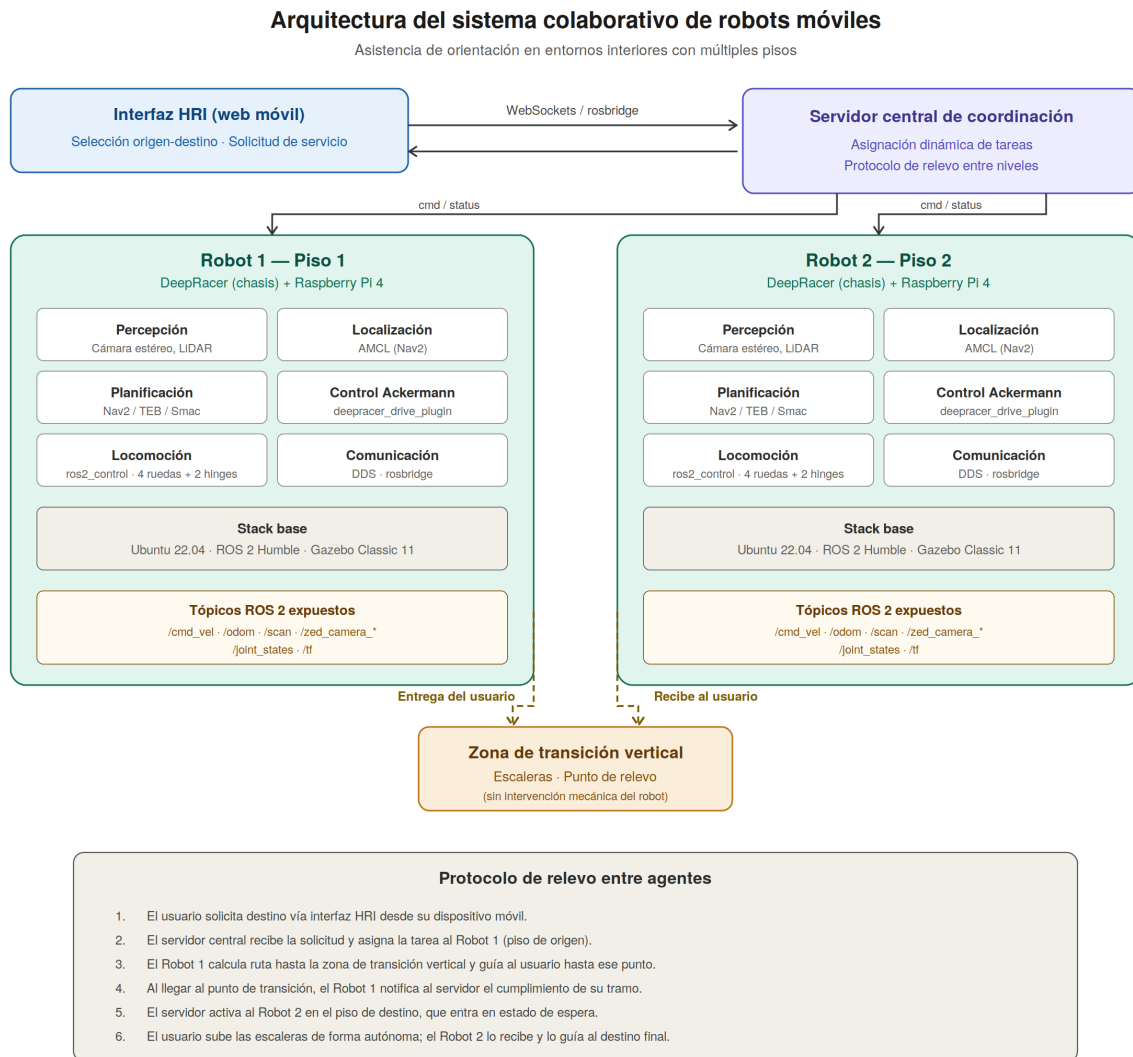


Figura 3.1. Arquitectura funcional del sistema colaborativo, tal como quedó modelada en el OE1. Ningún robot atraviesa la zona de transición: lo que la cruza es el usuario, y lo que se traspasa entre agentes es la responsabilidad de guiarlo.

Dos casillas de la figura se concretaron más tarde, y conviene decirlo para que el lector no las lea como definitivas. La de planificación nombra los candidatos que se consideraron en el diseño; cuál quedó y por qué está en el §3.2.3. Y los niveles aparecen como pisos 1 y 2 porque así se planteó el escenario. Al cambiar de sitio de pruebas pasaron a ser los pisos 3 y 4 (§3.2.5), y la arquitectura no cambió. Que no cambiara es justamente lo que se esperaba de ella.

3.1.1 El contrato de interfaces

Antes de escribir código se fijó un contrato: qué habla con qué y en qué formato. El sistema tiene tres piezas. Un **coordinador** central decide y manda. Dos **agentes**, uno por nivel, ejecutan. Y una **interfaz web** por la que el usuario pide el servicio.

La comunicación sigue tres reglas que no se rompen:

1. El coordinador manda a un agente **solo** mediante la acción de navegación de ROS 2. No publica velocidades directamente.
2. La interfaz habla **solo** con el coordinador. Nunca con los agentes.
3. El texto que ve el usuario lo redacta el coordinador. La interfaz lo muestra literal.

La tercera evita que la lógica de presentación se duplique en el navegador. Si el cliente interpretara el estado numérico para decidir qué mostrar, habría dos copias de la misma lógica y acabarían divergiendo.

3.1.2 El planificador, escrito sin ROS a propósito

El relevo es el aporte declarado del proyecto. Por eso se escribió como función pura: entran el catálogo de puntos y un par origen-destino, salen los tramos que hay que ejecutar. No importa el middleware ni mantiene estado.

La razón es de verificabilidad. Si el relevo viviera dentro del nodo, mezclado con clientes de acción, la única forma de comprobarlo sería levantar dos simuladores y mirar si el carro llega. Eso es lento y frágil. Como función pura se agota en milisegundos: **la prueba recorre 930 combinaciones de origen-destino y verifica cada plan, sin simulador.**

Esa prueba encontró defectos que una corrida no habría delatado. El más claro: cuando el origen o el destino *es* el punto de transferencia de su nivel, el plan mandaba al mismo robot dos veces a la misma pose. Un desplazamiento de cero metros no falla en Nav2 —devuelve éxito al instante— así que la simulación no lo habría mostrado.

3.1.3 La asignación de tareas

Cada nivel tiene un robot asignado, y esa correspondencia es un dato de configuración, no una constante del código. El coordinador elige el agente al aceptar la solicitud, según el nivel del origen y del destino.

En la taxonomía de Gerkey y Mataric [2] es una asignación centralizada e instantánea. Con dos agentes y el nivel decidiendo cuál responde, un esquema de subasta añadiría complejidad sin dar flexibilidad.

El planificador exige **exactamente un punto de transferencia por nivel** y falla con un mensaje explícito si encuentra dos. Hoy cada planta tiene una escalera y eso se cumple. La verificación está puesta para que el día que haya dos la decisión se tome a la vista, y no en silencio.

3.2 Plataforma robótica (OE2)

3.2.1 Por qué DeepRacer y no DonkeyCar

El OE2 habla de «dos vehículos tipo DonkeyCar». Los vehículos empleados son dos AWS DeepRacer, y el cambio conviene explicarlo porque el lector lo va a notar.

DonkeyCar y DeepRacer son la misma clase de plataforma: un chasis de radiocontrol a escala 1:18 con dirección Ackermann, un computador a bordo y software abierto. Lo que el objetivo fija —cinemática tipo automóvil, bajo costo, código modificable— se cumple igual con una que con otra. La diferencia es que la universidad ya tenía los dos DeepRacer, de modo que usarlos evitó construir hardware y dejó ese tiempo para el problema que el proyecto declara como suyo, que es la coordinación.

El cambio no sale gratis y hay que decirlo. El DeepRacer trae su propia pila de software, pensada para conducción autónoma por visión y para competencia, y buena parte del trabajo del §3.2.3 consistió en convivir con ella. Un DonkeyCar partiría de menos capas, pero habría que montarlo.

3.2.2 Nav2 sobre un vehículo que no gira sobre su eje

La plataforma sigue cinemática tipo automóvil. No es holonómica: no puede girar en el sitio. La configuración por omisión del marco de navegación de ROS 2 asume tracción diferencial, de modo que propone trayectorias que el vehículo no puede ejecutar.

Se sustituyeron dos de sus tres capas. La planificación global pasó a un planificador híbrido A* con modelo Reeds-Shepp, que admite marcha atrás y genera las maniobras de inversión que el vehículo necesita. El control de seguimiento pasó a un controlador de persecución pura regulada, con marcha atrás habilitada. Del árbol de comportamiento se quitó la recuperación que gira sobre el propio eje, porque el vehículo no puede hacerla.

3.2.3 El orden de puesta en marcha

Cada pieza de la cadena de navegación solo se puede diagnosticar si la anterior funciona. La guía oficial de puesta en marcha de Nav2 [29] lo plantea como una escalera de siete peldaños. De abajo arriba: transformaciones entre marcos, odometría, barrido del láser, mapa, localización, mapas de costos y, al final, planificador y controlador.

El proyecto la siguió en ese orden, y fue lo que permitió aislar el fallo del apartado siguiente. Sin la escalera, una odometría que no mide se confunde con un planificador que no planifica, porque los dos se manifiestan igual: el vehículo no llega. Con ella, el problema queda acotado a un peldaño.

El orden importa además en la ejecución. Los mapas de costos leen el mapa al configurarse. Si el servidor de mapas no está activo en ese instante, la capa estática queda vacía. Y entonces el planificador rechaza cualquier meta diciendo que el punto de partida está ocupado. El mensaje señala el origen, y la causa está tres peldaños más abajo.

3.2.4 La odometría, que fue el cuello de botella

El vehículo no tiene encoders de rueda. Su única fuente de odometría es la correspondencia entre barridos sucesivos del láser. Durante meses esa pieza no funcionó, y el proyecto tardó en entender por qué.

La causa no era de calibración sino de información. En un pasillo recto y de sección uniforme, cada barrido es prácticamente idéntico al anterior a lo largo del eje del corredor. El estimador devuelve un desplazamiento cercano a cero y *acierta*: desde el sensor no ha cambiado nada. Se midió sobre el edificio y el resultado fue contundente: en los pasillos disponibles la odometría registraba entre el 5 % y el 73 % del avance real.

La figura 3.2 muestra lo que eso produce. Las tres trayectorias son del mismo vehículo en el mismo hall. La localización se desplaza hasta más de dos metros respecto de donde el vehículo estaba de verdad, medido con flexómetro, y en una de las corridas salta fuera del recinto. La línea a trazos

marca otro hallazgo de esa misma sesión: la pared norte estaba modelada 0,84 m más al norte de lo que está.

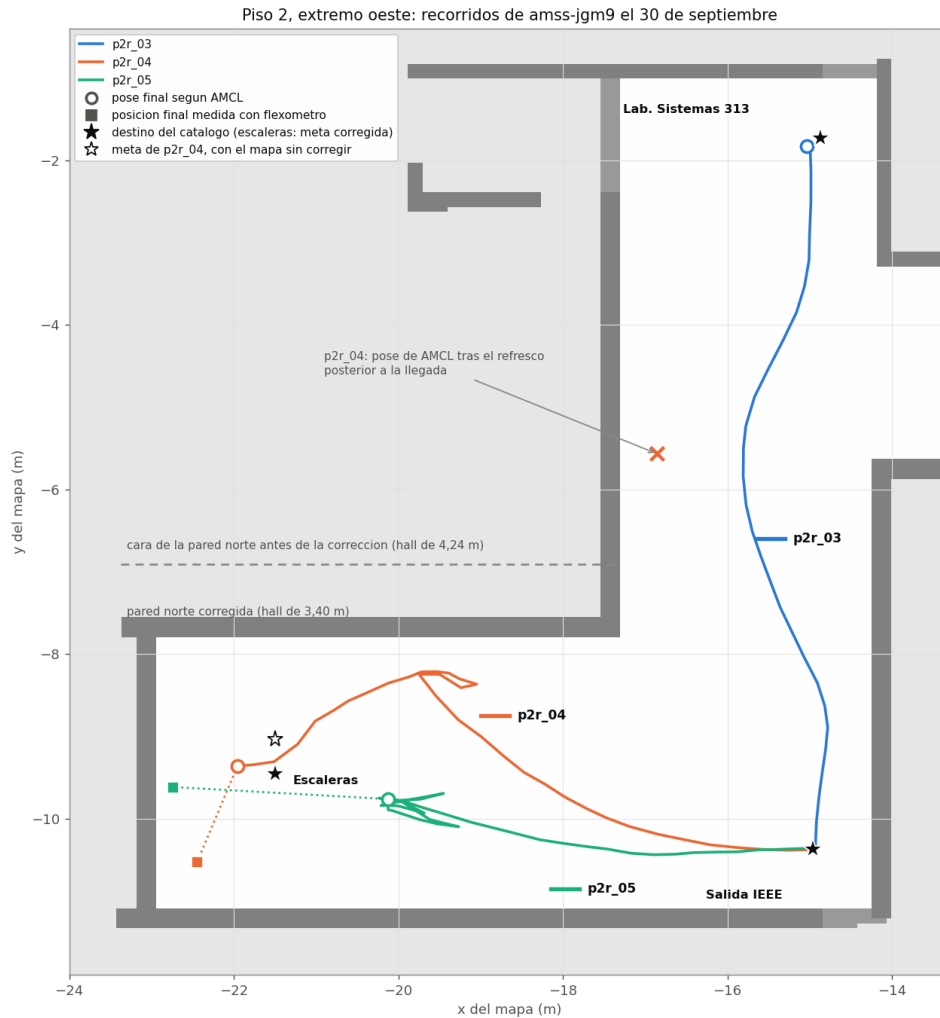


Figura 3.2. Tres recorridos en el hall del piso 2. El círculo es la pose que reporta la localización al terminar; el cuadrado, la posición real medida con flexómetro. La separación entre los dos es el error, y su causa está en la odometría, no en el algoritmo de localización.

La solución no fue afinar el algoritmo, porque no había nada que afinar. Fue cambiar de entorno. El criterio para elegirlo salió de la misma medida: hace falta estructura no uniforme dentro del alcance del sensor durante todo el recorrido.

Se levantaron con flexómetro dos plantas que la tienen. Entre las tres hornacinas de salón, los lockers, la papelera, el ascensor y la escalera suman ocho cambios de perfil en 25 m, uno cada tres metros. Con la misma odometría y sin tocar un parámetro, el error cayó a menos del 5 %.

3.2.5 Los mapas, derivados de medidas y no de SLAM

El proyecto intentó durante semanas construir el mapa con SLAM y no lo consiguió en ningún pasillo, por la misma razón del apartado anterior. La vía que funcionó fue otra: **levantar la planta con flexómetro y derivar de ese levantamiento tanto el mundo simulado como el mapa de navegación.**

Eso tiene una consecuencia que conviene subrayar. Simulación y edificio coinciden porque salen de la misma fuente, no porque se parezcan. El proyecto había dado esa correspondencia por supuesta, y al verificarla por primera vez resultó falsa: el modelo anterior daba un hall 0,84 m más ancho de lo que es, un 20 % de error.

La herramienta que genera las plantas incluye una comprobación que no se salta. Un pasillo mide lo mismo por sus dos lados, así que suma las dos paredes y avisa si no cierran dentro del 2 %. Esa comprobación encontró cinco errores antes de generar nada. Las dos plantas cierran al 0,32 % y al 0,62 %.

La figura 3.3 es el resultado para una de ellas. Lo que se ve no es un plano dibujado sino el mapa de ocupación con el que navega el vehículo, generado a partir de las medidas. Los entrantes de los salones, los lockers, el ascensor y la escalera son los ocho cambios de perfil que hacen observable el avance.

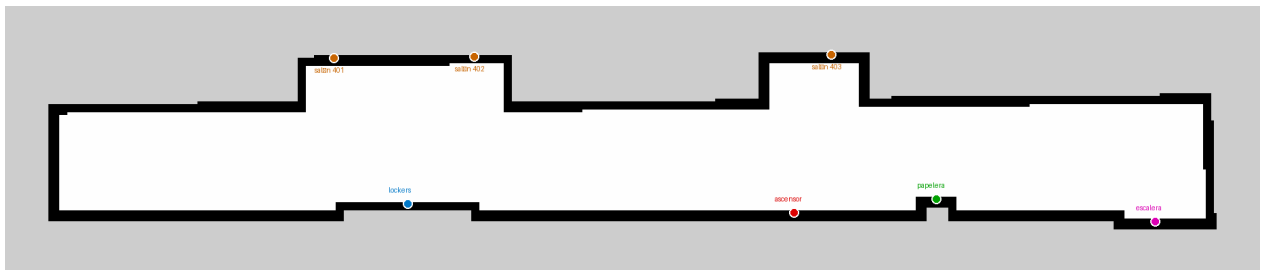


Figura 3.3. Mapa de navegación del piso 4, derivado del levantamiento con flexómetro. El pasillo corre 25 m del extremo norte al sur; los salones quedan al este, y el ascensor y la escalera al oeste.

3.2.6 La banda muerta de tracción, y lo que obliga a configurar

Al integrar la cadena de tracción apareció una característica de la plataforma que condiciona toda la aproximación al destino. **El vehículo no se mueve con velocidad comandada por debajo de 0,40 m/s.** El puente que traduce la orden de velocidad a la señal del variador la convierte en tracción nula, y lo hace sin emitir ningún aviso: desde fuera parece que la orden se ejecutó.

Eso forzó una decisión de configuración que conviene dejar escrita, porque tiene consecuencias. La velocidad mínima de aproximación del controlador se subió a ese mismo valor, 0,40 m/s. Sin ese ajuste el vehículo se detenía al entrar en la fase final del recorrido y se quedaba ahí, intentando moverse a una velocidad que su propia cadena anulaba.

El efecto es que **el vehículo se aproxima a la meta a 0,40 m/s y no puede ir más despacio**. No tiene régimen de aproximación fina. Es un límite de la plataforma, no del software, y no se corrige afinando el controlador: las dos salidas son bajar el umbral de arranque del vehículo —por mecánica o por calibración de los servos— o fijar la tolerancia de llegada en un valor compatible con la plataforma, con justificación previa. Las dos son decisiones, no ajustes, y por eso quedan planteadas en el capítulo de trabajos futuros.

3.3 Interfaz humano-robot (OE3)

3.3.1 Una página web, no una aplicación

El OE3 pide «una interfaz móvil». Lo que se construyó es una página web que se abre en el navegador del teléfono. Cumple el objetivo: es móvil porque se usa desde el teléfono del usuario. Y lo cumple mejor que una aplicación nativa. Un visitante ocasional —que es el usuario que el proyecto declara— no va a instalar nada para preguntar por un salón.

El requisito pedía que el usuario no instalara nada. La interfaz es una página web que se abre desde el navegador del teléfono. No usa recursos externos, porque debe funcionar sin salida a internet: en la red del edificio puede no haberla.

La comunicación con el middleware pasa por un puente que expone las acciones de ROS 2 como JSON sobre WebSocket. La librería de cliente más difundida del ecosistema no servía: su cliente de acciones corresponde a la generación anterior del middleware y es incompatible. La interfaz usa por eso un cliente propio, escrito directamente sobre el protocolo del puente.

3.3.2 Lo que el usuario ve

El usuario elige origen y destino de un catálogo con búsqueda por nombre, y pulsa un botón. A partir de ahí la página muestra el estado de la misión con el texto que redacta el coordinador.

Dos funciones se añadieron después de ver el sistema en uso:

Confirmación de cambio de piso. La secuencia de una misión entre niveles termina la etapa de transferencia cuando el robot del piso de destino llega a su escalera. Pero el robot no puede saber si *la persona* ya subió. Sin una pausa explícita, la misión podía declararse completada con el usuario todavía en el piso equivocado, y la instrumentación lo habría registrado como éxito.

Cancelación. El botón existía pero no hacía nada, y nadie lo había notado: el servidor de acción no registraba la función de cancelación, de modo que el middleware rechazaba toda solicitud por omisión y en silencio. Al corregirlo se añadió además el comportamiento que faltaba: al cancelar, el robot en curso regresa a su escalera antes de que la misión cierre.

3.4 Protocolo experimental (OE4)

El protocolo se escribió **antes** de instrumentar el sistema, y esa secuencia es deliberada. Fija las cuatro métricas con definición operativa, el criterio de éxito, el tamaño de muestra y una lista cerrada de causas de descarte con su techo.

Dos decisiones del protocolo merecen mención porque condicionan cómo se leen los resultados.

La primera: **el éxito se mide contra la odometría del vehículo, nunca contra el resultado que devuelve Nav2.** Una navegación puede terminar en *éxito* y haber dejado al robot lejos del destino. Aceptar ese resultado sería dejar que el sistema se evalúe a sí mismo.

La segunda: **las misiones se sortean.** Con pares elegidos a mano se corre el riesgo de probar justo los recorridos que funcionan.

Durante la escritura del analizador apareció un defecto que ninguna corrida habría revelado, y que se trata en el §4.3: uno de los criterios estaba formulado de manera que no podía dar resultado negativo.

4. RESULTADOS

Apartado en curso: la toma de datos cierra el 16 de octubre de 2026.

Las cifras de este capítulo son las disponibles al **5 de octubre de 2026**. Las dos mitades están en situaciones distintas y conviene no leerlas igual:

- El §4.1, **simulación**, está **cerrado**. La campaña de treinta misiones corrió el 5 de septiembre, el analizador la dictaminó válida y sus números no van a cambiar.
- El §4.2, **hardware**, está **sin redactar a propósito**. Las sesiones de campo continúan hasta el corte del **viernes 16 de octubre**, fijado en el acta de directores, después del cual no se admiten más datos. Reportar ahora unas cifras que van a cambiar obligaría a corregirlas después, así que la sección dice qué irá en ella y no adelanta ningún resultado.

El §4.3 trata de las limitaciones de la campaña de simulación, así que cierra con ella.

Los resultados llegan de dos campañas distintas. La primera, en simulación, con treinta misiones sorteadas. La segunda, sobre los vehículos reales en los pasillos del edificio. Se reportan por separado porque miden cosas distintas: la de simulación mide el sistema completo con relevo, la de hardware mide hasta dónde llega la plataforma física.

4.1 Evaluación en simulación

La campaña corrió treinta misiones sorteadas, quince de control dentro de un mismo nivel y quince entre niveles. El analizador dictaminó **campana válida**, con **cero descartes** contra un techo admitido del 20 %.

4.1.1 Tasa de éxito (RF-23)

Una misión es exitosa si cumple dos cosas. Que la pose final quede a 0,25 m o menos del destino, medida contra la odometría y no contra el resultado de Nav2. Y que la misión llegue a completarse sin pasar por fallo.

Tabla 4.1. Tasa de éxito de la campaña, con intervalo de confianza de Wilson.

	n	éxitos	tasa	IC 95 % (Wilson)
Campaña completa	30	26	86,7 %	70,3 – 94,7 %
Condición A — control	15	12	80,0 %	54,8 – 93,0 %
Condición B — inter-nivel	15	14	93,3 %	70,2 – 98,8 %

La afirmación defendible es que la tasa de éxito supera el 70 %, no que sea del 86,7 %. Con treinta misiones el intervalo mide 24 puntos porcentuales de ancho, y ese límite es de diseño: no se arregla trabajando más, se arreglaría con más muestra. El 86,7 % es el estimador puntual y se reporta siempre junto a su intervalo.

Los dos intervalos de las condiciones se solapan, así que **la comparación entre ellas no sostiene ninguna conclusión**. Conviene decirlo porque el estimador puntual apunta en contra de la intuición: la condición con relevo salió mejor que la de control.

4.1.2 Los cuatro fallos son un solo modo

Las cuatro misiones fallidas quedaron a 0,295, 0,284, 0,311 y 0,347 m del destino, contra una tolerancia de 0,25 m. Las cuatro se quedaron *cortas*, ninguna se pasó ni terminó en otro sitio.

Eso no es un fallo de coordinación: es el límite de precisión de la aproximación final. El error de llegada del conjunto tiene mediana **0,106 m** y recorrido de 0,019 a 0,347 m.

4.1.3 Tiempo de respuesta (RF-21) y de asignación (RF-22)

El tiempo de respuesta se reporta por escalones y no con media y desviación, por una razón de instrumento:

Tabla 4.2. Tiempo de respuesta, en escalones del reloj de simulación.

t respuesta	0,1 s	0,2 s	0,3 s	0,4 s
misiones	10	15	4	1

Los cuatro valores son múltiplos exactos de 100 ms, que es el tic del reloj de simulación. **La resolución del instrumento vale entre el 25 % y el 100 % de la magnitud medida**. Una mediana

de 0,2 s significa «uno o dos tics», no «doscientos milisegundos». Dar media y desviación sugeriría una precisión que el reloj no entrega.

Lo que sí se sostiene es la cota, que además es lo que le importa a un usuario: **el robot arranca en menos de medio segundo.**

El tiempo de asignación quedó por debajo de 100 ms en las treinta. Como esa cifra cae tres órdenes de magnitud por debajo de un tic del reloj, se caracterizó aparte en banco: mediana entre 154 y 175 μ s, máximo 306 μ s.

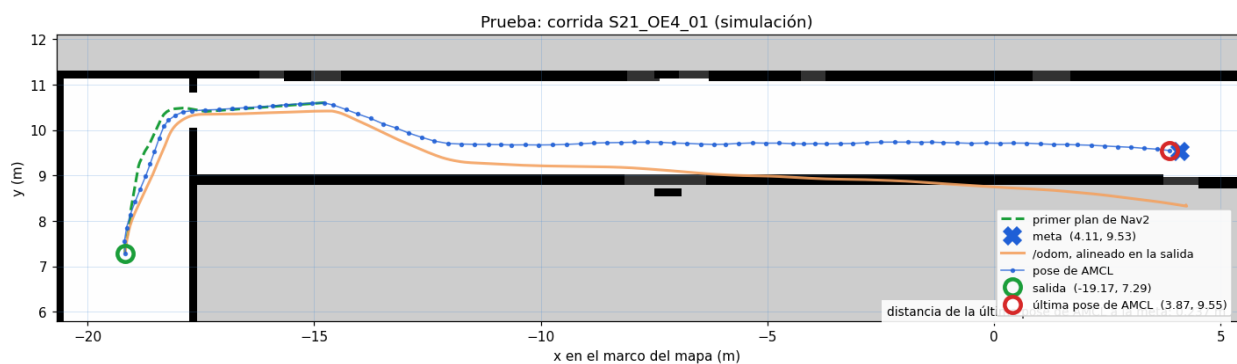


Figura 4.1. Una misión de la campaña en simulación. Se superponen el primer plan que calcula el navegador, la pose estimada a lo largo del recorrido y la odometría cruda. La separación creciente entre las dos últimas es la deriva que la localización corrige contra el mapa.

4.2 Evaluación sobre hardware

Apartado en curso: sección abierta: la toma de datos cierra el 16 de octubre de 2026.

Esta sección **queda pendiente de redacción** y por eso no se presenta ningún resultado en ella. No es un apartado olvidado: es el único frente del proyecto que sigue tomando datos, y escribirlo ahora obligaría a corregirlo después.

Lo que irá aquí, cuando cierre:

1. **La odometría del vehículo** (compuerta G-2 del acta de directores): error de la estimación de avance contra medida con flexómetro, sobre los recorridos de los pisos 3 y 4.
2. **La navegación**: evidencia de que el planificador traza y el controlador sigue sobre el vehículo real, a partir del registro de cada corrida.
3. **La precisión de llegada** (compuerta G-3): distancia medida entre la parada del vehículo y el destino del catálogo, contra la tolerancia de 0,5 m fijada por el director.

Hay sesiones de campo ya realizadas y sus medidas están registradas en el repositorio del proyecto, pero **no se reportan todavía como resultado**: G-2 y G-3 no están declaradas en el acta, y esa declaración corresponde al equipo en el corte del **viernes 16 de octubre de 2026**. Lo que haya ese día es lo que se defiende.

4.3 Limitaciones de la medición

Tres límites afectan a cómo deben leerse los resultados anteriores. Se declaran aquí porque callarlos sería presentar como medida lo que no lo es.

4.3.1 La continuidad entre niveles es un invariante, no un resultado

La continuidad del servicio es la variable de respuesta principal del objetivo 4, y su criterio binario se cumplió en las treinta misiones. **Ese resultado no es evidencia de nada**, y la razón es estructural.

El criterio exige que dentro de la ventana de la misión el coordinador nunca esté en estado inactivo y nunca tenga el agente sin asignar. Los dos únicos puntos del código que producen esos valores ocurren *antes* de que la ventana se abra. No existe ejecución posible que lo incumpla: son cero incumplimientos de cero posibles.

Un criterio que no puede dar resultado negativo no aporta evidencia, por muchas veces que se verifique. La corrección no fue inventar una ruta de fallo. Eso habría sido instrumentar el experimento para que pareciera más informativo. Lo que se hizo fue **declarar la continuidad como invariante estructural del diseño** y apoyar la afirmación en una magnitud que sí varía: el hueco temporal del relevo. En las quince misiones entre niveles tuvo mediana de 0,1 s y máximo de 0,2 s.

4.3.2 El intervalo de confianza es ancho por diseño

Ya se dijo en la tabla 4.1 y se repite porque es el límite más fácil de olvidar al redactar: con treinta misiones, **lo defendible es una cota inferior**, no un valor puntual. Aumentar la confianza exige más muestra, no más trabajo.

4.3.3 El mapa de la campaña leía como libre lo desconocido

El mapa con que corrió la campaña declaraba un umbral que hacía que **10 908 celdas desconocidas —el 14,5 % del mapa— se cargaran como espacio libre**. Con ello, las dos salvaguardas que debían

impedir planificar sobre lo no explorado quedaban desactivadas de hecho.

El alcance de la afirmación, y no más: esto **no invalida el veredicto**. Los cuatro fallos fueron de precisión de llegada, no de trazado, y el pasillo por el que se navegó está mapeado como libre de verdad. Lo que no se sabe es si alguna trayectoria cruzó una de esas celdas, y es comprobable sobre los registros conservados. El defecto está corregido desde el 25 de septiembre.

CONCLUSIONES

Apartado en curso: conclusiones parciales: dos de los cuatro objetivos siguen abiertos.

Estas conclusiones se emiten el **5 de octubre de 2026**, con el proyecto todavía en curso, y por eso **no están todas**. Se escriben solo las que descansan sobre algo cerrado:

- **OE1 y OE3** están verificados, así que sus conclusiones son definitivas.
- **OE2 y OE4** dependen de la campaña física, que cierra el **16 de octubre**. Sus conclusiones quedan abiertas, y aquí se dice qué falta para poder escribirlas.
- El **objetivo general** se enuncia, porque descansa en la campaña de simulación, que ya cerró. Lo que falta no es el esquema sino su verificación sobre hardware.

Sobre OE1, la arquitectura funcional. El objetivo se cumplió. El sistema tiene un contrato de interfaces escrito, treinta y cuatro requisitos numerados con su prueba, y una capa de coordinación que ejecuta el relevo entre niveles. Escribir el planificador como función pura, sin depender del middleware, resultó más valioso de lo previsto. Permite verificar 930 combinaciones de origen y destino en milisegundos. Y encontró defectos que una corrida en el simulador no habría delatado, como planes que mandaban al mismo robot dos veces a la misma pose.

Sobre OE2, la plataforma robótica. Conclusión abierta. Lo que sí se puede afirmar hoy es de construcción, no de desempeño: los dos vehículos están integrados y Nav2 quedó reconfigurado para cinemática tipo automóvil, con el planificador, el controlador y el árbol de comportamiento sustituidos por los que esa cinemática admite (§3.2). El juicio sobre si el objetivo se cumple **queda pendiente** del cierre de la campaña física, el 16 de octubre, y se decidirá sobre las compuertas G-2 y G-3 del acta de directores.

Sobre OE3, la interfaz humano-robot. El objetivo se cumplió. La interfaz funciona desde el navegador de un teléfono, sin instalación y sin salida a internet, y se operó una misión completa con relevo desde un dispositivo real. Dos funciones se añadieron después de ver el sistema en uso. Una es la confirmación de cambio de piso, que impide dar por completada una misión con el usuario en el nivel equivocado. La otra es la cancelación, que no funcionaba: el servidor de acción rechazaba las solicitudes en silencio.

Sobre OE4, la evaluación. Cumplido en simulación; abierto sobre hardware. La campaña de treinta misiones sorteadas dio veredicto válido con cero descartes, y permite afirmar que **la tasa de éxito supera el 70 %** con 95 % de confianza. Esa mitad del objetivo está cerrada y sus cifras no van

a cambiar. La otra mitad, la evaluación sobre los vehículos reales, sigue tomando datos hasta el 16 de octubre y se concluye entonces (§4.2).

Sobre el objetivo general. El esquema de relevo **es viable**. Dos robots dedicados cada uno a un nivel pueden repartirse el guiado de una persona que cambia de piso. Ninguno atraviesa la zona de transición vertical, y el edificio no se modifica. Eso quedó demostrado en simulación, con una campaña medida y con veredicto válido. **Lo que queda abierto no es el esquema sino su verificación sobre hardware**, que es de lo que se ocupa la campaña física en curso. El esquema de relevo, en cambio, ya no está a prueba.

Una conclusión de método, que el trabajo defiende con el mismo peso. La parte más costosa del proyecto no fue construir el sistema sino averiguar qué estaba midiendo de verdad. Tres episodios lo muestran. Una métrica valía cero por construcción hasta que se saneó. Un criterio de aceptación no podía dar resultado negativo, así que sus treinta verificaciones en verde no eran evidencia. Y la correspondencia entre el modelo del edificio y el edificio real se daba por supuesta desde agosto, y resultó falsa en un 20 % la primera vez que se midió. Los tres se encontraron preguntando si el criterio podía fallar, y ninguno habría aparecido corriendo más experimentos.

TRABAJOS FUTUROS

Apartado en curso: lista sujeta a lo que arroje la campaña física.

Esta lista se cierra cuando cierre la toma de datos, el 16 de octubre de 2026. Las cuatro líneas que siguen salen de límites ya medidos y no van a desaparecer; lo que puede ocurrir es que las sesiones que faltan añadan alguna más.

Las líneas que siguen salen de límites concretos que el trabajo encontró y documentó, no de una lista genérica de mejoras.

Resolver la aproximación fina del vehículo. Es la limitación que condiciona la precisión de llegada, y está localizada y medida (§3.2.6). El vehículo no se mueve por debajo de 0,40 m/s: la cadena de tracción traduce esa velocidad a cero. Hay dos vías. Una es de hardware —calibrar los servos o cambiar la relación de transmisión— y daría al vehículo un régimen lento del que hoy carece. La otra es de criterio: fijar la tolerancia de llegada a un valor compatible con la plataforma, con justificación previa y no a la vista de los resultados.

Añadir una fuente de odometría independiente del láser. La odometría actual deduce el movimiento comparando barridos, y eso falla por construcción en pasillos uniformes. El proyecto lo resolvió eligiendo un entorno con estructura, pero esa solución limita dónde puede operar el sistema. Una unidad inercial, o odometría visual con las cámaras que los vehículos ya llevan, daría información de avance que no depende de la geometría del sitio.

El ascensor como segunda vía de transición. Hoy el cambio de piso se hace siempre por la escalera, y el ascensor es un destino más. Admitir los dos exige que el planificador elija entre dos puntos de transferencia por nivel, presumiblemente el más cercano al punto anterior. El código ya nombra esa decisión y falla explícitamente cuando encuentra dos, de modo que el punto de extensión está identificado.

Resolver una esquina. Todos los recorridos medidos fueron sobre tramos rectos. El guiado real tiene esquinas. El controlador demostró mover la dirección, pero no se ha probado que el vehículo resuelva un cambio de rumbo grande en un pasillo estrecho. Un intento de media vuelta en un corredor de 2,3 m no fue posible.

Ampliar la muestra de la campaña. Con treinta misiones el intervalo de confianza mide 24 puntos porcentuales. Ese ancho es de diseño y solo se reduce con más repeticiones. Una campaña mayor

permitiría además comparar las dos condiciones experimentales, cosa que hoy no se sostiene porque sus intervalos se solapan.

Operar con los dos vehículos en el mismo grafo. Las pruebas de hardware se hicieron con un vehículo cada vez, porque los dos publican en los mismos nombres de tópico y una orden alcanza a ambos. Separarlos por espacio de nombres está diseñado y probado en el computador, pero no ejercitado en campo, y es condición para la demostración del sistema completo.

REFERENCIAS BIBLIOGRÁFICAS

- [1] M. Jiménez Muñoz, «Arquitectura cognitiva para integración de usuarios humanos dentro de un sistema de mapas multinivel en una plataforma robótica asistencial,» Tesis de mtría., Universidad Politécnica de Madrid, Dept. Ing. Industrial, Madrid, España, 2024.
- [2] B. P. Gerkey y M. J. Matarić, «A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems,» *The International Journal of Robotics Research*, vol. 23, n.º 9, págs. 939-954, 2004. DOI: 10.1177/0278364904045564.
- [3] G. T. Varghese et al., «Multi-Robot System for Mapping and Localization,» en *8th International Conference on Robotics and Automation Engineering (ICRAE)*, 2023, págs. 1-6. DOI: 10.1109/ICRAE59816.2023.10458504.
- [4] T. Kim, G. Kang, D. Lee y D. H. Shim, «Development of an Indoor Delivery Mobile Robot for a Multi-Floor Environment,» *IEEE Access*, vol. 12, págs. 1-12, 2024. DOI: 10.1109/ACCESS.2024.3381489.
- [5] Z. Song, S. Lou, R. Tan y C. Lv, «Robot-Elevator Interaction through IoT Devices for Autonomous Robot Multi-Floor Transition,» en *10th International Conference on Electrical Engineering, Control and Robotics (EECR)*, 2024, págs. 1-6. DOI: 10.1109/EECR60807.2024.10607245.
- [6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette y W. Woodall, «Robot Operating System 2: Design, Architecture, and Uses in the Wild,» *Science Robotics*, vol. 7, n.º 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074.
- [7] K. Omer y A. Monteriu, «Multi-Layer Robotic Controller for Enhancing the Safety of Mobile Robot Navigation in Human-Centered Indoor Environments,» *Frontiers in Robotics and AI*, vol. 12, págs. 1-15, 2025. DOI: 10.3389/frobt.2025.00001.
- [8] B. R. Nair, «Collaborative Perception in Multi-Robot Systems: Case Studies in Household Cleaning and Warehouse Operations,» en *6th International Conference on Robotics and Computer Vision (ICRCV)*, 2024, págs. 1-6. DOI: 10.1109/ICRCV60186.2024.00012.
- [9] A. Sojan, C. Thomas, B. Raj, S. A. Simon, S. K. John y A. A. Jose, «Autonomous Indoor Navigator,» en *11th International Conference on Smart Computing and Communication (ICSCC)*, 2025, págs. 1-6. DOI: 10.1109/ICSCC66177.2025.11233830.
- [10] M. Bojarski et al., «End to End Learning for Self-Driving Cars,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Preprint, arXiv:1604.07316, Daejeon, South Korea, 2016.

- [11] L. H. Zain, «Real-Time Obstacle Avoidance for a Mobile Robot Using CNN-Based Sensor Fusion,» en *International Conference on Robotics and Automation (ICRA)*, Preprint, arXiv:2509.08095, 2025.
- [12] J. López Velásquez, G. A. Acosta Amaya y J. A. Jiménez Builes, «Enhancing Mobile Robot Navigation: Integrating Reactive Autonomy through Deep Learning and Fuzzy Behavior,» *Journal of Intelligent & Robotic Systems*, vol. 110, n.º 2, págs. 45-58, 2024. DOI: 10.1007/s10846-024-02045-2.
- [13] E. Roszkowska, P. Makowski-Czerski y L. Janiec, «Multi-Level Control for Multiple Mobile Robot Systems,» en *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, págs. 1-7. DOI: 10.1109/ICRA48891.2023.10160734.
- [14] B. Rudrusamy et al., «Multi-Robot Systems Using Broadcast Control Framework with Visual-Based Tracking,» en *7th International Conference on Mechanical Engineering and Robotics Research (ICMERR)*, 2022, págs. 1-6. DOI: 10.1109/ICMERR56497.2022.10097802.
- [15] X. Xu, S. A. Prieto y B. García de Soto, «A Modular ROS Framework for Cooperative Multi-Robot Task Allocation in Digital Environments,» *MDPI Sensors*, vol. 25, n.º 4, págs. 1-20, 2025. DOI: 10.3390/s25040001.
- [16] J. Zhan, X. Yu y T. Westerlund, «Mejorando la resiliencia de los sistemas multirrobóticos basados en ROS 2 con Kubernetes: un estudio de caso sobre el posicionamiento,» *MDPI Sensors*, vol. 25, n.º 16, pág. 5067, 2025. DOI: 10.3390/s25165067.
- [17] A. K. Srivastava, J. M. Levin y P. Dames, «DELIVER: A System for Coordinated Multi-Robot Pickup and Delivery Using Relay Planning,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, págs. 1-8.
- [18] A. E. Piguave Toala y D. S. Ronquillo Manosalvas, *Diseño e implementación de un sistema multi-robot de código abierto para ambientes colaborativos en ROS2*, Proyecto integrador, Fac. Ing. Electrónica, Escuela Superior Politécnica del Litoral, Guayaquil, Ecuador, 2023.
- [19] S. Luo, Q. Yu y C. Zhu, «“Elevator or Stairs”: A Layered Reinforcement Learning Approach for Robot Multi-Floor Navigation in High-Traffic Scenarios,» en *International Conference on Advanced Computing and Intelligent Robotics Applications (ACIRA)*, 2025, págs. 1-8. DOI: 10.1109/ACIRA67680.2025.11334740.
- [20] J. Chen et al., «LITE: A Learning-Integrated Topological Explorer for Multi-Floor Indoor Environments,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, págs. 1-8. DOI: 10.1109/IROS60139.2025.11246317.

- [21] Z. Gong et al., «Stairway to Success: An Online Floor-Aware Zero-Shot Object-Goal Navigation Framework via LLM-Driven Coarse-to-Fine Exploration,» *IEEE Robotics and Automation Letters*, vol. 11, n.º 3, págs. 1-8, 2026. DOI: 10.1109/LRA.2026.3655265.
- [22] A. Li y H. Gurocak, «Modified Bug Algorithm with Proximity Sensors to Reduce Human-Cobot Collisions,» en *9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, págs. 1-6. DOI: 10.1109/ICARA56516.2023.10126056.
- [23] B. Kehoe, S. Patil, P. Abbeel y K. Goldberg, «A Survey of Research on Cloud Robotics and Automation,» *IEEE Transactions on Automation Science and Engineering*, vol. 12, n.º 2, págs. 398-409, 2015. DOI: 10.1109/TASE.2014.2376492.
- [24] N. Tahir y R. Parasuraman, «Edge Computing and Its Application in Robotics: A Survey,» *MDPI Sensors*, vol. 25, n.º 2, págs. 1-25, 2025. DOI: 10.3390/s25020001.
- [25] I. Malavolta et al., «Computation Offloading for Ground Robotic Systems Communicating over WiFi,» *Empirical Software Engineering*, vol. 28, n.º 3, págs. 1-35, 2023. DOI: 10.1007/s10664-022-10268-0.
- [26] Departamento Nacional de Planeación, «Documento CONPES 3975: Política Nacional para la Transformación Digital e Inteligencia Artificial,» Consejo Nacional de Política Económica y Social, Bogotá D.C., Colombia, inf. téc., 2019.
- [27] Departamento Nacional de Planeación, «Documento CONPES 4069: Política Nacional de Ciencia, Tecnología e Innovación 2022–2031,» Consejo Nacional de Política Económica y Social, Bogotá D.C., Colombia, inf. téc., 2021.
- [28] ICONTEC, «NTC 1486: Documentación. Presentación de tesis, trabajos de grado y otros trabajos de investigación,» Instituto Colombiano de Normas Técnicas y Certificación, Bogotá D.C., Colombia, inf. téc., 2022.
- [29] Nav2 Project, *First-Time Robot Setup Guide*, Documentación de Nav2, <https://docs.nav2.org>, Consultado en septiembre de 2026.